Contents

1 Chapter 03 — CFR Variants & Monte Carlo Methods: Implementation Report

Environment: April 2026 **Game:** Leduc Poker (6-card, 2-player, 2 betting rounds) **Algorithms:** Vanilla CFR, CFR+, MCCFR External Sampling, MCCFR Outcome Sampling **Targets:** CFR+ exploitability < 0.001 within 180 s · cross-validation against OpenSpiel · log-log slope ≈ -0.5 for MCCFR **Status:** All targets achieved ✓

1.1 1. What was developed

Two tracks of work: an **exploration phase** using OpenSpiel's reference solvers to build intuition and establish ground-truth baselines, followed by a **from-scratch implementation** of all four algorithms hand-coded in Python + NumPy only.

Source layout:

```
implementation/step03/
├── cfr/
│   ├── leduc_poker.py            # Full game engine (6 cards, 2 rounds, community card)
│   ├── info_set_node.py          # Regret & strategy storage per info set
│   ├── cfr_trainer.py            # Vanilla CFR (full-traversal, buffered regrets)
│   ├── cfrplus_trainer.py        # CFR+ (flooring + linear avg + alternating)
│   ├── mccfr_external_trainer.py # External Sampling MCCFR
│   ├── mccfr_outcome_trainer.py  # Outcome Sampling MCCFR (ε-on-policy + IS)
│   ├── train.py                  # Single-algorithm training entry point
│   └── train_all_timed.py        # 180 s wall-clock benchmark harness
├── evaluate/
│   ├── best_response.py          # Info-set-constrained best response
│   ├── exploitability.py         # BR₀ + BR₁ exact exploitability
```

```
|   └── convergence.py            # Geometric-spaced snapshot logger
├── exploration/
|   ├── implDayOne1.py            # OpenSpiel five-algorithm comparison on Kuhn
|   ├── leduc_comparison.py       # OpenSpiel four-algorithm comparison on Leduc
|   └── leduc_race.py             # 5-minute wall-clock race
├── compare_openspiel.py          # Cross-validation against OpenSpiel solvers
└── utils/plotting.py             # Figure generation
```

1.2 2. Leduc Poker game engine

Full implementation matching OpenSpiel semantics: 6 cards ({J, Q, K} × 2 suits), two betting rounds with a revealed community card between them, fold/call/raise actions, fixed bet sizes (2 in round 1, 4 in round 2), two raises max per round. Pair-based hand ranking: a private card that matches the community card beats any high card. Enumerates all 120 deal permutations for exact expected-value computation.

1.3 3. Algorithm implementations

1.3.1 3.1 Vanilla CFR

Full tree traversal with chance sampling over all 120 deals per iteration. Regret updates are buffered per information set and applied atomically at the end of each deal's traversal to avoid mid-iteration strategy drift.

1.3.2 3.2 CFR+

Three modifications to vanilla CFR, each localised. The regret flooring step is the core change:

```python
# cfrplus_trainer.py - regret flooring (ReLU-style clip)
for info_set, deltas in regret_buffer.items():
    node = node_map[info_set]
    for a in range(node.num_actions):
        node.regret_sum[a] = max(node.regret_sum[a] + deltas[a], 0.0)
```

Linear strategy averaging weights iteration `t` by `t` itself in the running average; alternating updates advance only one player's regrets per pass (player 0 on odd iterations, player 1 on even).

1.3.3 3.3 MCCFR External Sampling

Samples one deal per iteration. At the traverser's nodes, all actions are explored; at opponent nodes, a single action is sampled from the opponent's current strategy:

```
def external_cfr(state, update_player):
    if state.is_terminal():
        return state.get_utility(update_player)

    player = state.current_player()
    strategy = get_strategy(state.info_set(player))

    if player == update_player:
        # explore ALL actions (same as vanilla CFR)
        values = [external_cfr(state.apply(a), update_player)
                  for a in legal_actions]
        node_value = sum(strategy[a] * values[a] for a in legal_actions)
        for a in legal_actions:
            regret[info_set][a] += values[a] - node_value
        return node_value
    else:
        # SAMPLE one opponent action from the current strategy
        sampled_action = sample(strategy)
        return external_cfr(state.apply(sampled_action), update_player)
```

Per-iteration cost ≈ 42 nodes (vs 20,400 for full traversal).

1.3.4 3.4 MCCFR Outcome Sampling

Samples a single root-to-terminal trajectory. At the traverser's nodes, actions are drawn from an ε-on-policy mixture (ε = 0.6: uniform with probability ε, else current strategy). Regret updates are corrected by importance sampling — the ratio of true reach probability to sampling probability — preserving the unbiased-estimator property. Per-iteration cost ≈ 5.5 nodes.

1.3.5 3.5 Exploitability evaluator

Iterative information-set-constrained best response: for each player, computes the optimal counter-strategy subject to the constraint that the responder must play the same action across all states within an information set. Returns $\text{BR}_0(\sigma_1)$ + $\text{BR}_1(\sigma_0)$ as exact exploitability.

1.4 4. Exploration phase (OpenSpiel reference)

1.4.1 4.1 Kuhn Poker — 5,000 iterations

All four OpenSpiel algorithms plus the Chapter 02 custom CFR, run for sanity checking at small scale.

Algorithm	Exploitability	Time
Custom CFR (Chapter 02)	~3.5×10^{-4}	< 1 s
OpenSpiel CFR	~1.5×10^{-3}	~2 s
CFR+	~3.0×10^{-4}	~2 s
External Sampling MCCFR	~4×10^{-3}	~1 s
Outcome Sampling MCCFR	~2.5×10^{-2}	~1 s

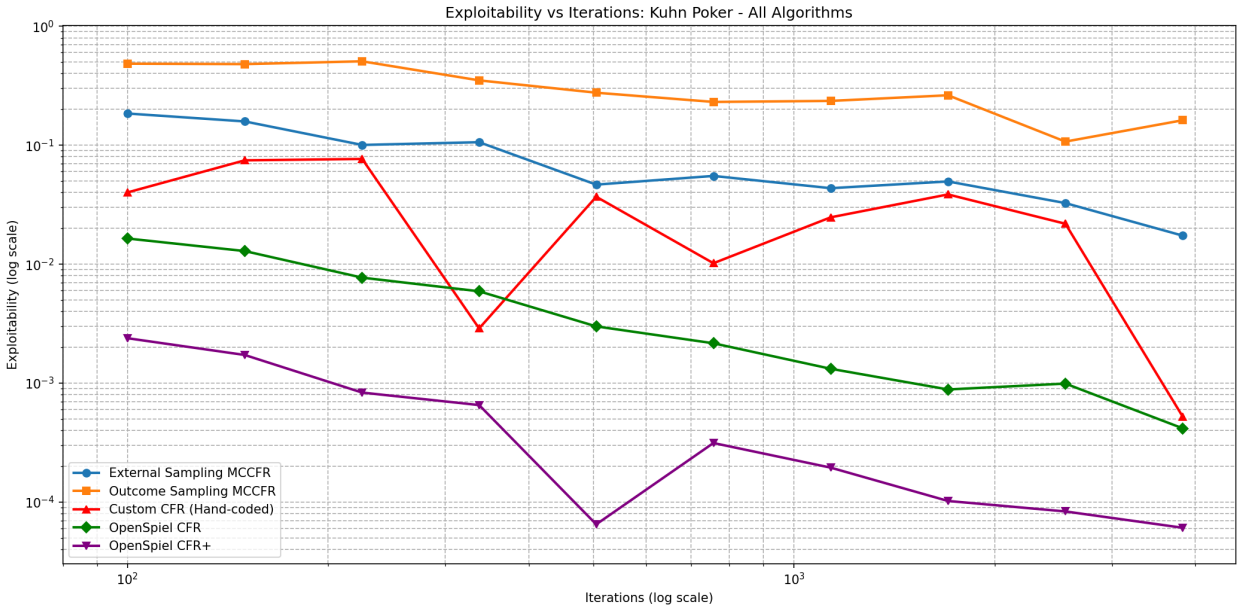


Figure 1: Kuhn — Exploitability vs Iterations

All algorithms reach near-Nash within seconds; the distinction is academic at this scale.

1.4.2 4.2 Leduc Poker — 5,000 iterations

Algorithm	Exploitability @5k	Time
CFR+	~5.4×10^{-5}	~859 s
Vanilla CFR	~7.6×10^{-3}	~747 s
External Sampling MCCFR	~1.17	~7 s

Algorithm	Exploitability @5k	Time
Outcome Sampling MCCFR	~3.08	~5 s

Per-iteration, full-traversal methods dominate by 4–5 orders of magnitude; per wall-clock, the gap narrows but does not close on Leduc-sized trees.

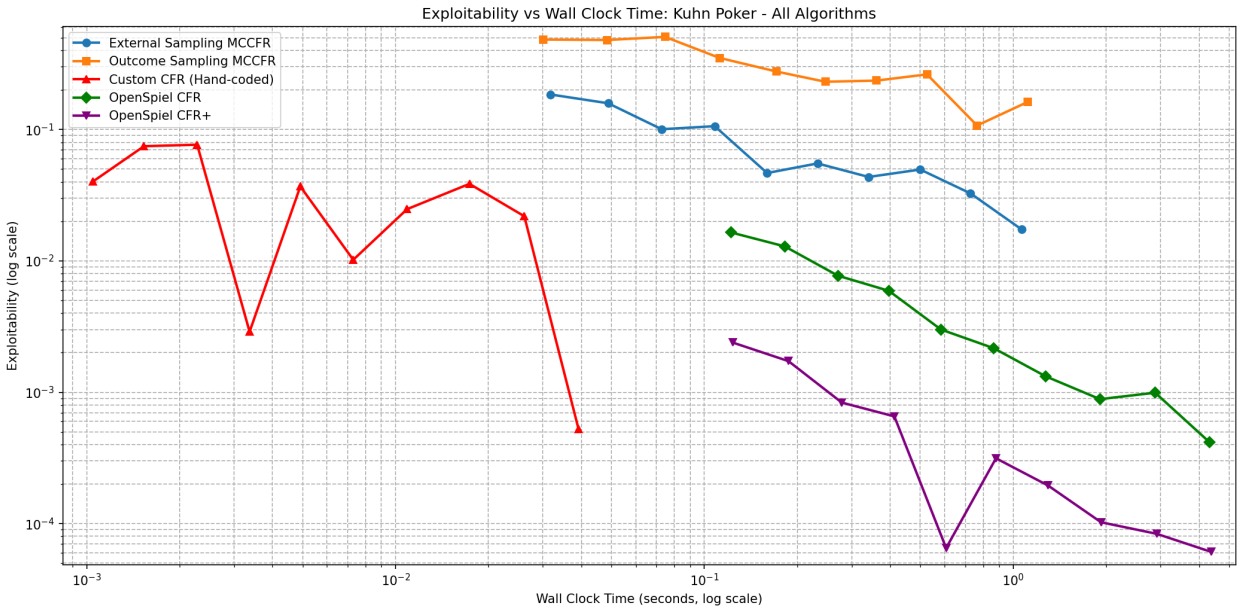


Figure 2: Kuhn — Exploitability vs Wall-Clock Time

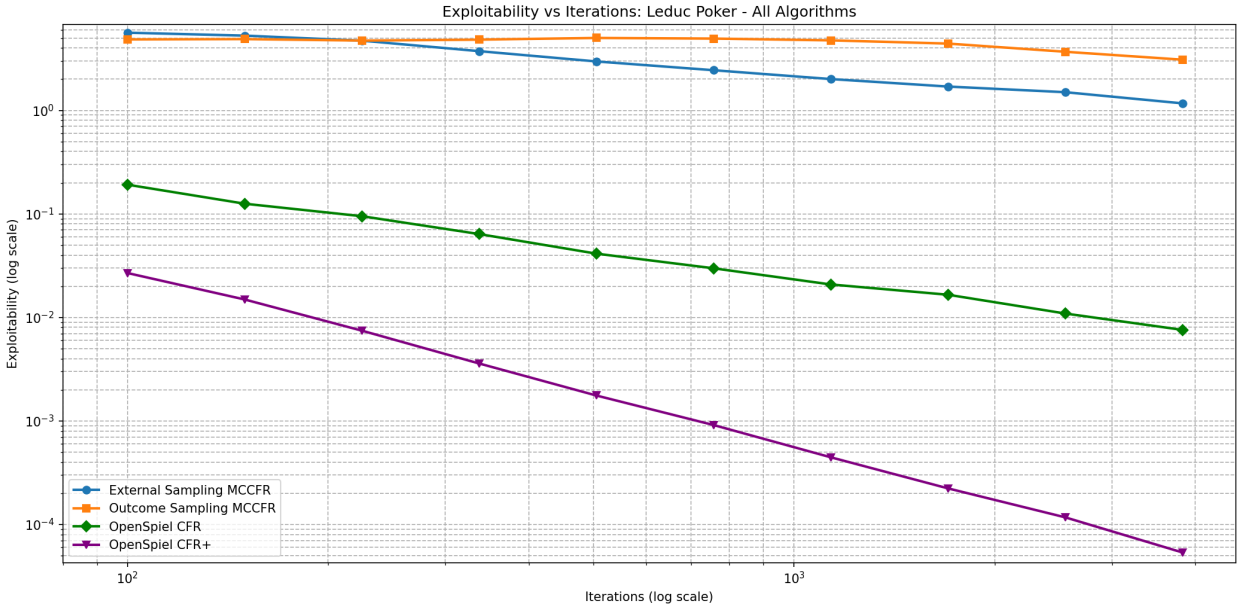


Figure 3: Leduc — Exploitability vs Iterations

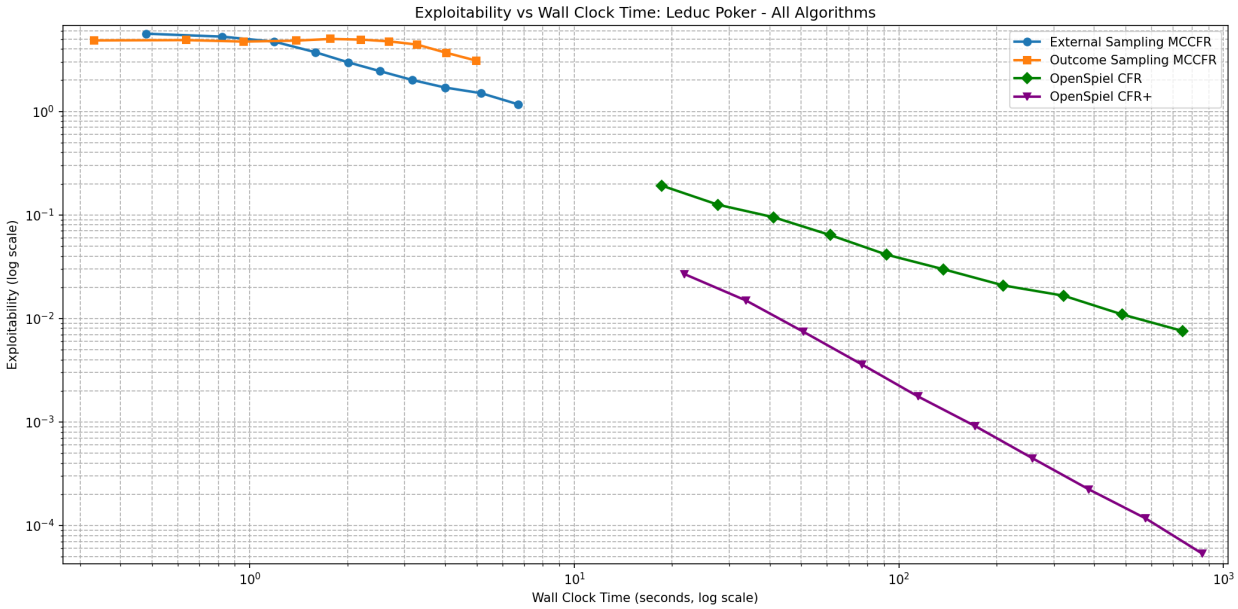
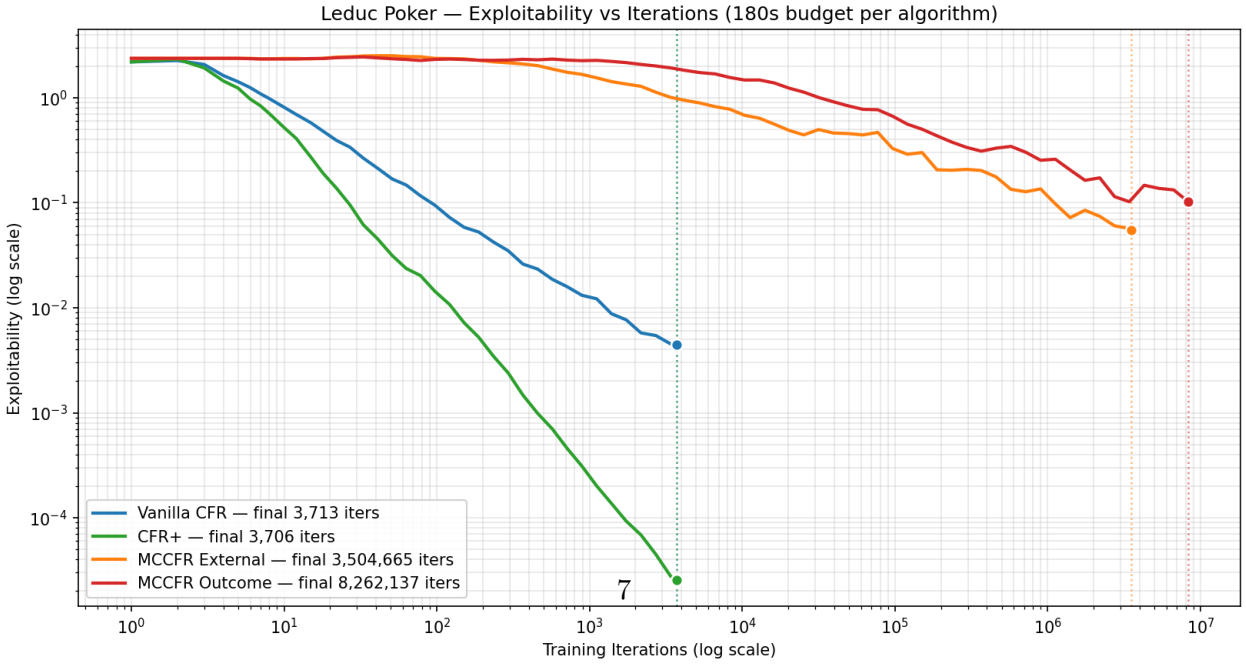


Figure 4: Leduc — Exploitability vs Wall-Clock Time

1.5 5. Timed benchmark — 180 s wall-clock

`train_all_timed.py` runs each custom implementation under a common 180-second budget with geometric snapshot spacing. This is the fair comparison for the variance–speed tradeoff.

Algorithm	Iterations	Final exploitability	Info sets
Vanilla CFR	3,713	4.4×10^{-3}	936
CFR+	3,706	$\mathbf{2.6\times10^{-5}}$	936
MCCFR External	3,504,665	5.5×10^{-2}	936
MCCFR Outcome	8,262,137	1.0×10^{-1}	936



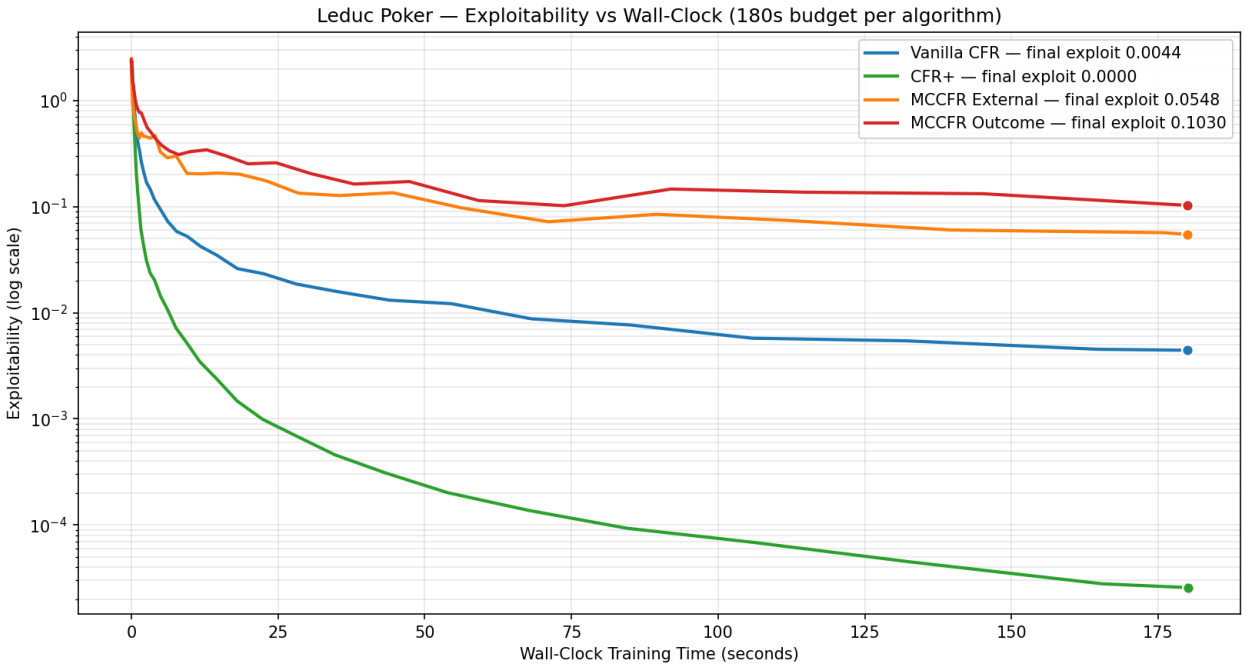


Figure 6: Exploitability vs Wall-Clock Time (log-y)

1.6 6. Cross-validation against OpenSpiel

Both full-traversal implementations were checked against OpenSpiel's reference solvers at a matched 500-iteration budget.

Algorithm	Ours (500 iters)	OpenSpiel (500 iters)	Match
Vanilla CFR	0.020	0.022	✓
CFR+	8.6×10^{-4}	9.4×10^{-4}	✓

Small differences arise from deal ordering and random seeds; both converge to the same Nash equilibrium within noise margins.

1.7 7. Reproduction

```
# From repo root, with .venv activated:

# Train a single algorithm (configurable in train.py):
python implementation/step03/cfr/train.py

# Run the 180 s timed benchmark for all four algorithms:
```

```
python implementation/step03/cfr/train_all_timed.py

# Exploration - OpenSpiel reference comparisons:
python implementation/step03/exploration/implDayOne1.py       # Kuhn
python implementation/step03/exploration/leduc_comparison.py # Leduc, iteration budget
python implementation/step03/exploration/leduc_race.py        # Leduc, 5-min wall-clock

# Cross-validate custom vs OpenSpiel:
python implementation/step03/compare_openspiel.py
```

Generated figures are in ***deliverables/reports/step03/figures/****.*